```
0000 with TEXT_IO;¶use  TEXT_IO;¶¬¬¬----------¶procedure¬¬¬BLOCK_SAVE¶is¬¬¬----------¶¶  BLOCK_WIDTH¬¬: constant NATURAL¬:= 150;¬ -- caracteres utiles par
     ligne¶  NUM_PREFIX_LEN¬¬: constant NATURAL¬:= 5;¬ -- "NNNN " en tete¶  BLOCKS_PER_NUMBER¬¬: constant NATURAL¬:= 10;¬ -- numerotation tous les N blocs¶
       BLOCKS_PER_SECTION¬: constant NATURAL¬:= 50;¬ -- ligne¬vide tous¬les N blocs¶¶  LIST_FILE_STR¬¬: STRING(¬1 .. 128 );¶  LIST_FILE_LEN¬¬: NATURAL¬¬:=
     0;¶  LIST_FILE¬¬: FILE_TYPE;¶¶  PROCESSED_FILE_STR¬: STRING(¬1 .. 256 );¶  PROCESSED_FILE_STR_LEN¬: NATURAL¬:= 0;¶¶¶¬¬----------¶  function¬
     STRIP_LEADING_DOTDOT¬( PATHED_FILE_NAME :STRING )¬return STRING¶  is¬¬----------¶    -- Supprime les¬sequences¬initiales¬"../" et "./" d'un
     chemin,¶    -- de¬sorte que¬"../../src/foo/bar.adb" devienne "src/foo/bar.adb".¶    ID¬¬¬: NATURAL¬:= PATHED_FILE_NAME'FIRST;¶  begin¶    loop¶      i
     f¬ID + 2 <=¬PATHED_FILE_NAME'LAST¶        and then PATHED_FILE_NAME( ID .. ID+2 ) =¬"../"¶      then¶        ID := ID + 3;¶      elsif  ID + 1¬<= PATH
     ED_FILE_NAME'LAST¶        and then PATHED_FILE_NAME( ID .. ID+1 ) =¬"./"¶      then¶        ID := ID + 2;¶      else¶        exit;¶      end¬if;¶    e
     nd loop;¶    return PATHED_FILE_NAME( ID .. PATHED_FILE_NAME'LAST );¶¶  end¬STRIP_LEADING_DOTDOT;¶¬----------¶¶¶¬¬----------¶  function¬BL
     OCKED_NAME¬( SRC_NAME :STRING )¬return STRING¶  is¬¬----------¶    -- Transforme le nom de fichier source en nom¬de fichier bloc,¶    -- en¬remplaca
0010 nt l'avant-derniere lettre de l'extension¬par 'B'.¶    -- Par exemple : "toto.adb" -> "toto.Badb" ; "toto.ads"¬-> "toto.Bads".¶    -- Preserve le chem
     in qui¬precede le nom de fichier.¶    -- Si¬le nom est trop court pour avoir une extension de¬3 caracteres,¶    -- ajoute simplement ".B"¬en suffixe.¶
       begin¶    if  SRC_NAME'LENGTH >= 4¬and then¬SRC_NAME(¬SRC_NAME'LAST - 3 )¬= '.'¶    then¶      return SRC_NAME( SRC_NAME'FIRST .. SRC_NAME'LAST - 3
     )¶¬   & "B"¶¬   & SRC_NAME( SRC_NAME'LAST - 2 .. SRC_NAME'LAST¬);¶    else¶      return SRC_NAME & ".B";¶    end if;¶¶  end¬BLOCKED_NAME;¶¬----------
     -¶¶¶¬¬----------¶  function¬FOUR_DIGITS¬( N :NATURAL )¬return STRING¶  is¬¬----------¶    -- Convertit un¬entier 0..9999 en STRING de longueur 4,¬re
     mbourre¬a gauche¶    -- par des zeros. Par exemple 23 devient "0023".¶    RESULT¬: STRING(¬1 .. 4 )¬:= "0000";¶    VAL¬¬: NATURAL¬¬:= N;¶  begin¶    f
     or  ID in reverse 1 .. 4¬ loop¶      RESULT( ID ) := CHARACTER'VAL( CHARACTER'POS( '0' ) +¬VAL mod 10 );¶      VAL¬:= VAL / 10;¶    end loop;¶    retu
     rn RESULT;¶¶  end¬FOUR_DIGITS;¶¬----------¶¶¶¬¬----------¶  procedure¬BLOCK_FILE¬( FILE_IN, FILE_OUT¬:in FILE_TYPE )¶  is¬¬----------¶    PROCESS_IN_
     BUFFER¬: STRING(¬1 .. 512 );¶    LINE_LEN¬¬: NATURAL¬¬:= 0;¶    PROCESS_OUT_BUFFER¬: STRING(¬1 .. BLOCK_WIDTH );¶    OCAR¬¬: NATURAL¬¬:= 0;¶    BLOCK_
     COUNT¬¬: NATURAL¬¬:= 0;¶¶¶¬¬----------¶    procedure¬FLUSH_BLOCK¶    is¬¬----------¶      -- Emet le bloc courant¬PROCESS_OUT_BUFFER(¬1 .. OCAR¬) su
0020 r FILE_OUT,¶      -- precede du¬prefixe de 5 caracteres (numero ou espaces) et¶      -- eventuellement precede d'une ligne vide en separateur de secti
     on.¶    begin¶      -- Ligne de separation tous les BLOCKS_PER_SECTION blocs, au debut de¶      -- chaque nouvelle section (donc avant les blocs 50, 1
     00, 150, ...).¶      if¬BLOCK_COUNT /= 0 and then BLOCK_COUNT mod BLOCKS_PER_SECTION = 0  then¶        NEW_LINE( FILE_OUT );¶      end¬if;¶¶      -- P
     refixe : numero tous les BLOCKS_PER_NUMBER blocs,¬espaces sinon.¶      if¬BLOCK_COUNT mod BLOCKS_PER_NUMBER = 0  then¶        PUT( FILE_OUT, FOUR_DIGI
     TS( BLOCK_COUNT )¬& " " );¶      else¶        declare¶¬SPACES¬:constant¬STRING( 1¬.. NUM_PREFIX_LEN )¬:= (others => ' ');¶        begin¶¬PUT( FILE_OUT
     , SPACES );¶        end;¶      end¬if;¶¶      -- Corps du bloc.¶      PUT_LINE( FILE_OUT, PROCESS_OUT_BUFFER( 1 .. OCAR ) );¶¶      BLOCK_COUNT := BLO
     CK_COUNT + 1;¶      OCAR := 0;¶¶    end¬FLUSH_BLOCK;¶¬----------¶¶¶  begin¶¬¬¬----------¶¬¬¬PROCESS_ALL_LINES:¶    while¬ not END_OF_FILE( FIL
     E_IN )  loop¶      GET_LINE( FILE_IN, PROCESS_IN_BUFFER, LINE_LEN );¶¬¬----------¶¬¬PROCESS_ONE_LINE:¶      for¬ ICAR in 1 .. LINE_LEN  loop¶
         OCAR := OCAR + 1;¶¶        if  PROCESS_IN_BUFFER( ICAR ) =¬ASCII.HT¬then¶¬PROCESS_OUT_BUFFER(¬OCAR ) :=¬CHARACTER'VAL( 172 );¶¶        elsif  CHA
     RACTER'POS(¬PROCESS_IN_BUFFER( ICAR ) ) in 32 .. 126  then¶¬PROCESS_OUT_BUFFER(¬OCAR ) :=¬PROCESS_IN_BUFFER( ICAR );¶¶        else¶¬OCAR := OCAR - 1;¶
0030 ¶        end if;¶¶        if  OCAR = PROCESS_OUT_BUFFER'LAST  then¶¬FLUSH_BLOCK;¶        end if;¶¶      end¬loop¬PROCESS_ONE_LINE;¶¬----------¶
     ¶      OCAR := OCAR + 1;¶      PROCESS_OUT_BUFFER( OCAR ) := CHARACTER'VAL( 182 );¶      if¬OCAR = PROCESS_OUT_BUFFER'LAST  then¶        FLUSH_BLOCK;¶
       end¬if;¶¶    end loop¬PROCESS_ALL_LINES;¶¬----------¶¶    if  OCAR > 0  then¶      FLUSH_BLOCK;¶    end if;¶¶  end¬BLOCK_FILE;¶¬---------
     --¶¶¶begin¶  PUT( "NOM DU FICHIER LISTE DE TEXTES SOURCES : " );¶  GET_LINE( LIST_FILE_STR, LIST_FILE_LEN );¶  OPEN( LIST_FILE, IN_FILE, LIST_FILE_STR
     ( 1 .. LIST_FILE_LEN ) );¶¶  while  not END_OF_FILE( LIST_FILE )  loop¶    GET_LINE( LIST_FILE, PROCESSED_FILE_STR, PROCESSED_FILE_STR_LEN );¶    if
     PROCESSED_FILE_STR_LEN /= 0 and¬then PROCESSED_FILE_STR( 1 ) /= '#'  then¶¬¬----------¶¬¬PROCESS_ONE_FILE:¶      declare¶        PROCESSED_FILE¬
     : FILE_TYPE;¶        BLOCKED_FILE¬: FILE_TYPE;¶        OPEN_OK¬¬: BOOLEAN¬:= FALSE;¶        CREATE_OK¬¬: BOOLEAN¬:= FALSE;¶      begin¶        begin¶¬
     OPEN( PROCESSED_FILE, IN_FILE, PROCESSED_FILE_STR( 1 .. PROCESSED_FILE_STR_LEN ) );¶¬OPEN_OK := TRUE;¶        exception¶¬when NAME_ERROR =>¶¬PUT_LINE(
     ¬"FICHIER SOURCE INEXISTANT " &  PROCESSED_FILE_STR( 1 .. PROCESSED_FILE_STR_LEN¬) );¶        end;¶¶      if  OPEN_OK¬ then¶¬declare¶¬REL_PATH¬:cons
     tant¬STRING¶¬¬¬:= STRIP_LEADING_DOTDOT( PROCESSED_FILE_STR( 1 ..¬PROCESSED_FILE_STR_LEN ) );¶¬OUT_FILE_STR¬:constant¬STRING¬:= "./BLK_SRC/" & BLOCKED_
0040 NAME( REL_PATH );¶¬begin¶¬begin¶¬  CREATE(¬BLOCKED_FILE, OUT_FILE, OUT_FILE_STR );¶¬  CREATE_OK := TRUE;¶¬  PUT_LINE( "PROCESSING : " &¬REL_PATH );¶¬e
     xception¶¬  when NAME_ERROR =>¶¬    PUT_LINE( "FICHIER SORTIE NAME_ERROR " & OUT_FILE_STR¶¬¬    & " (creer le repertoire destination avant de relancer
     )" );¶¬  when STATUS_ERROR¬=>¶¬    PUT_LINE( "FICHIER SORTIE STATUS_ERROR " & OUT_FILE_STR¬);¶¬  when USE_ERROR =>¶¬    PUT_LINE( "FICHIER SORTIE USE_
     ERROR " & OUT_FILE_STR¶¬¬    & " (repertoire inexistant ou droits insuffisants)" );¶¬end;¶¶¬if  CREATE_OK  then¶¬  BLOCK_FILE( PROCESSED_FILE,¬BLOCKED
     _FILE );¶¬  CLOSE( BLOCKED_FILE );¶¬end if;¶¬end;¶¶¬CLOSE( PROCESSED_FILE );¶      end if;¶¶      end¬PROCESS_ONE_FILE;¶¬----------¶    end if
     ;¶  end loop;¶¶  CLOSE( LIST_FILE );¶¶exception¶  when NAME_ERROR =>¶    PUT_LINE( "FICHIER LISTE INEXISTANT " &  LIST_FILE_STR(¬1 .. LIST_FILE_LEN )
     );¶end¬BLOCK_SAVE;¶
```